# mahi-gpu

8-thread SIMT GPGPU core for SkyWater sky130

Datasheet and Architecture Reference Manual

| | |
|---|---|
| **Document revision** | v1.0 — **preliminary** |
| **Date** | 2026-10-21 |
| **Design source** | repository commit `dadec7f`, directory `ASIC/src/rtl` |
| **Top module** | `gpu_asic_top` |
| **Target process** | SkyWater sky130A, `sky130_fd_sc_hd` |

**Status of this document**

This manual describes the RTL *as implemented*. It deliberately states what is **not** yet true of the chip: the **mini profile** has a routed layout that passes the checks available in the repository, but the full-size profile has never been routed (Chapter 5), the SRAM macros are timed at the typical corner only and are black boxes in LVS, nothing has been simulated on the gate-level netlist of this design, and barriers require warp-uniform control flow (a `SYNC` while diverged faults). See Chapter 6 (Errata) before relying on any number in Chapter 5.

# Contents


**1 Overview** **1**

1.1 Executive summary . . . 1
1.2 Key characteristics . . . 1
1.3 Conventions . . . 2

**2 System architecture and memory map** **3**

2.1 Block diagram . . . 3
2.2 Host interface: Wishbone slave . . . 3
2.2.1 Register map . . . 3
2.2.2 Internal Device Control Register (DCR) block . . . 3
2.3 Pinout, packaging and Caravel integration . . . 4
2.3.1 Pinout . . . 4
2.3.2 Packaging . . . 4
2.3.3 Caravel user-project wrapper . . . 4
2.4 Address spaces . . . 5
2.5 Reset and launch sequence . . . 5

**3 Core microarchitecture** **7**

3.1 Pipeline . . . 7
3.2 Datapath . . . 7
3.3 Warp scheduler . . . 7
3.3.1 Divergence handling (SIMT stack) . . . 8
3.3.2 Call/return stack . . . 8
3.4 Floating-point unit . . . 8
3.4.1 Scoreboard and hazards . . . 9
3.4.2 Register-file write arbitration . . . 9
3.5 Memory subsystem . . . 9
3.5.1 Load/store unit and coalescing . . . 9
3.5.2 Banked shared memory . . . 9
3.5.3 Caches and victim write buffer . . . 10
3.6 Exceptions . . . 10
3.7 Performance monitoring unit . . . 10

**4 Instruction set architecture** **11**

4.1 Instruction formats . . . 11
4.2 Register file conventions . . . 11
4.3 Instruction reference . . . 11
4.3.1 Integer arithmetic and logic . . . 12
4.3.2 Floating point (single precision) . . . 12
4.3.3 Compare, branch and call . . . 12
4.3.4 Memory, shared memory and atomics . . . 13
4.3.5 Synchronisation . . . 13

4.4 Programming example . . . 13
4.5 Architectural restrictions and hazards . . . 14

**5 Verification and physical implementation 15**
5.1 Verification methodology . . . 15
5.2 Regression results . . . 15
5.3 Mutation testing . . . 16
5.4 Bugs found and fixed during verification . . . 16
5.5 Physical implementation . . . 17
5.5.1 Flow and configuration . . . 17
5.5.2 Results of the final mini layout (`flow/runs/place_1029_1604`) . . . 17
5.5.3 Static timing across PVT corners . . . 18
5.5.4 Outstanding sign-off items . . . 18

**6 Errata and known limitations 19**

**A PMU event map 20**

**B Provenance and how to re-check this document 21**

Chapter 1

# Overview

## 1.1 Executive summary

**mahi-gpu** is a compact general-purpose GPU core built around the SIMT execution model. One *core* runs **2 warps** of **4 lanes** each (8 hardware threads); every lane has its own integer ALU, single-precision FMA unit, 32-entry register file, program counter and return stack. Warps are interleaved by a round-robin scheduler on a classic five-stage in-order pipeline (IF, ID, EX, MEM, WB) with single-cycle instruction fetch. As in production GPUs, there is no operand-forwarding network: an issue-time scoreboard holds back an instruction whose source register is still being produced, and another warp issues instead. A 5-stage pipelined IEEE-754-style FMA unit executes `FADD`, `FSUB`, `FMUL` and `FMA` under a per-warp register scoreboard. Memory is reached through a banked shared memory, an instruction cache, a 2-way write-through L1 data cache, and a 4-way write-back L2 cache with sector dirty bits and a victim write buffer. The host controls the device through a 32-bit Wishbone slave.

The ASIC build (`-DTARGET_ASIC`) is a size-reduced profile of a configurable design that also targets FPGA and simulation; all sizes in this manual are for the ASIC profile unless stated otherwise.

## 1.2 Key characteristics

| Characteristic | Value |
|---|---|
| Process / library | SkyWater sky130A, `sky130_fd_sc_hd` (high density), 1.8 V nominal |
| Supply corners analysed | 1.60 V/100 °C (ss), 1.80 V/25 °C (tt), 1.95 V/−40 °C (ff) |
| Flow | Yosys (via sv2v) → OpenROAD (floorplan, place, CTS, route) → Magic/KLayout, driven by LibreLane (§5.5) |
| Clock constraint (target) | 25 MHz (40 ns period), single clock `clk`, synchronous reset `reset` |
| Cores / warps / lanes | 1 core / 2 warps / 4 lanes = **8 threads** |
| Pipeline | 5-stage in-order (IF, ID, EX, MEM, WB); single-cycle fetch on I-cache hits; no forwarding (issue-time scoreboard); FPU: 5-stage, fixed latency |
| Register file | 32 × 32-bit per thread; `R0`–`R28` general, `R29`–`R31` read-only |
| Instruction word | 32 bit, fixed width; 37 opcodes defined (0–36) |
| Program memory | 4 K words (12-bit word address), external, 128-bit block interface |
| Global data memory | 16 K words (14-bit word address), external, 128-bit block interface |
| Shared memory | **64 words** (256 B), 4 banks × 16 words |
| L1 I-cache / L1 D-cache | 16 lines each; 16-byte lines (4 words); I: direct-mapped, D: 2-way, write-through |
| L2 cache | 64 lines (16 sets × 4 ways), 16-byte lines, 4 sectors/line with dirty bits, PLRU, write-back; 4-entry victim write buffer |
| Host interface | 32-bit Wishbone slave, single-cycle ack; **108 signal pins** (excl. power) |
| Synthesised logic (current RTL) | mini profile (LibreLane synthesis): 95 652 cells, 14 634 flip-flops, 1.07 $mm^2$ before placement; full profile (Yosys/ABC sizing script, `sky130_fd_sc_hd`): 226 744 cells, 23 964 flip-flops, 1.43 $mm^2$; plus 0.85 $mm^2$ for three SRAM macros (§5.5) |
| Die size | mini: 2.4 mm × 2.4 mm (routed); full: 3.0 mm × 3.0 mm configured, never routed |
| Package | none of its own: intended as a Caravel user project (§2.3); the harness ships as QFN-64 (9 mm × 9 mm), 60-ball WCSP or bare die |

## 1.3 Conventions

Numbers: `0x` prefix hexadecimal. Bit ranges `[hi:lo]` are inclusive. Addresses in the memory system are *word* addresses unless stated. “Lane” and “thread” are used interchangeably within a warp. Source citations use the form `file.sv:line` relative to `ASIC/src/rtl`. The words *limitation* and *erratum* refer to the register in Chapter 6 (IDs **L1**–**L14**).

Chapter 2

# System architecture and memory map

## 2.1 Block diagram

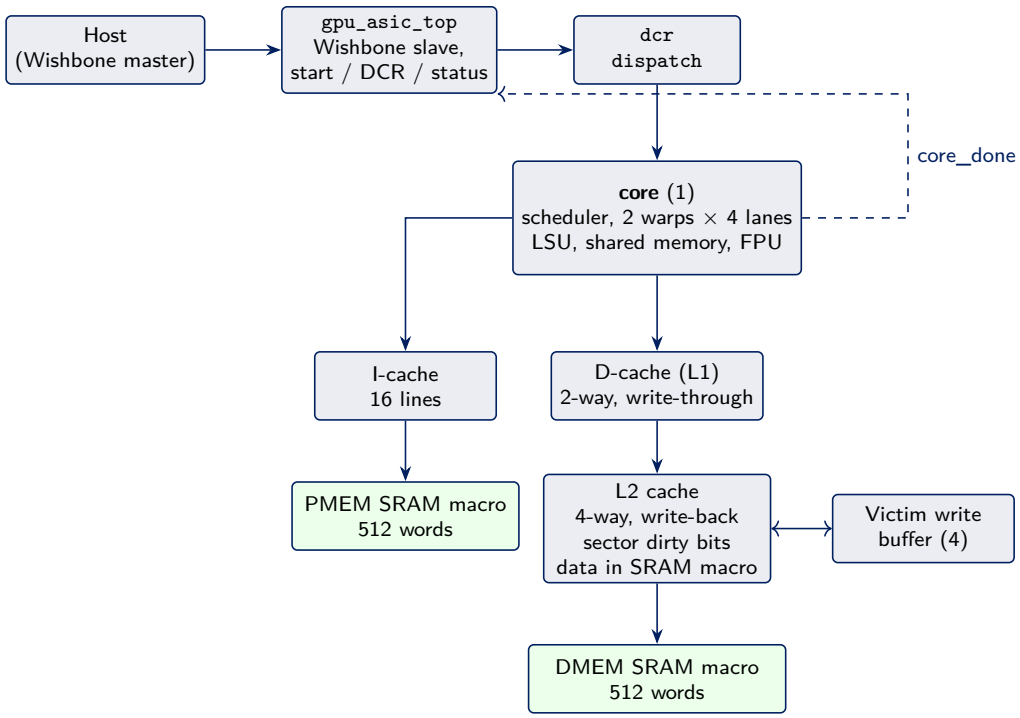



Figure 2.1: System block diagram of the ASIC profile. Green boxes are on-chip OpenRAM SRAM macros (a third, inside the L2 cache, holds its data array); the host reaches PMEM and DMEM through the Wishbone window.

The I-cache has its own path to the program-memory controller and does *not* pass through the L2. Both memory interfaces use a native 128-bit block protocol (valid/ready/address/data/strobe) through `controller.sv`. On chip, `gpu_mem_subsys` terminates them on two 512×32 SRAM macros (one block = four sequential word accesses). An AXI4 adaptor exists in the repository but is **not** instantiated in the ASIC top level.

## 2.2 Host interface: Wishbone slave

`gpu_asic_top` is a 32-bit Wishbone classic slave with a registered, single-cycle acknowledge (`wb_ack_o` rises one clock after a request with both `wb_cyc_i` and `wb_stb_i` set).

### 2.2.1 Register map

The base address is `0x3000_0000`. `wb_adr_i[31:12]` selects the page: `0x30000` is the control/status page, `0x30001` the memory window; every other address is acknowledged, reads as 0 and ignores writes. Every access is acknowledged (a window access that is refused still returns an ack). While the GPU runs, window accesses are refused: reads return `0xBAD0_BAD0`, writes are dropped and STATUS[3] is set (erratum L16).

### 2.2.2 Internal Device Control Register (DCR) block

The DCR (`dcr.sv`) has an 8-bit address / 8-bit data write port. **Only address `0x00` is reachable from the chip pins**: the Wishbone wrapper always drives address `0x00` (erratum L13). The remaining registers exist for integration and simulation.

| Signal | Dir | Width | Function |
|---|---|---|---|
| clk, reset | in | 1, 1 | clock, synchronous reset |
| wb_cyc_i, wb_stb_i, wb_we_i | in | 1 each | cycle, strobe, write enable |
| wb_sel_i | in | 4 | byte select (memory-window writes only) |
| wb_adr_i | in | 32 | address (bits [31:12] select the page, [7:0] the register) |
| wb_dat_i | in | 32 | write data |
| wb_dat_o | out | 32 | read data (registered) |
| wb_ack_o | out | 1 | acknowledge |
| gpu_done_pin | out | 1 | mirrors the done status bit |
| gpu_fault_pin | out | 1 | mirrors STATUS[4] (a warp faulted; sticky until cleared), intended for an in |

Table 2.1: Pins of gpu_asic_top: 108 signal pins in total (73 inputs, 35 outputs), excluding power.

| Address | Access | Name | Description |
|---|---|---|---|
| 0x3000_0000 | W | CTRL | [0] START (level; stays set until written 0); [1] SOFT_RESET (8 cycles; clears DCR, scheduler and all caches, keeps SRAM contents; drops START; other bits ignored); [15:8] THREAD_COUNT (written to internal DCR 0x00) |
| 0x3000_0000 | R | CTRL | {16'b0, THREAD_COUNT[7:0], 7'b0, START} |
| 0x3000_0004 | R/W1C | STATUS | [0] DONE; [1] BUSY; [2] GPU_OOB (sticky: data access outside the 512 words); [3] HOST_BLOCKED (sticky); [4] FAULT (sticky: a warp faulted); write 1 to clear [2]/[3]/[4] |
| 0x3000_0008 | R | FAULT_INFO | {PC[15:0], WARP[7:0], 4'b0, CAUSE[3:0]} of the first fault since STATUS[4] was last clear; CAUSE 1 = division by zero, 2 = bad address, 3 = SYNC while diverged |
| 0x3000_1000–17FF | R/W | PMEM | 512 words = GPU instruction words 0..511, byte strobes honoured |
| 0x3000_1800–1FFF | R/W | DMEM | 512 words = GPU data words 0..511, byte strobes honoured |

Table 2.2: Wishbone register map as implemented (gpu_asic_top.sv:27-38).

## 2.3 Pinout, packaging and Caravel integration

### 2.3.1 Pinout

The power pins are vccd1 (1.8 V) and vssd1 (present with USE_POWER_PINS). In the routed mini layout (§5.5) the 108 signal pins are placed by the flow along the die edges; there is no fixed pin order, so a different run may move them. A fixed placement comes from the integration target: the Caravel user area fixes every pin position.

### 2.3.2 Packaging

The GPU is not a packaged part on its own. It is built to be a **Caravel user project**: the Caravel harness (management RISC-V SoC, padframe, housekeeping) provides the package and the pads. Efabless listed Caravel as a 64-pin QFN (9 mm × 9 mm, 0.5 mm pitch), a 60-ball WCSP, or bare die; ChipFoundry now runs these shuttles, so the current offer must be confirmed with the operator. No GPIO pad is driven by the GPU (io_oeb = 1), so the package pinout is entirely Caravel's.

### 2.3.3 Caravel user-project wrapper

gpu_caravel_top (src/rtl/gpu_caravel_top.sv) wraps gpu_asic_top with the port names of Caravel's user_project_wrapper (2920 µm × 3520 µm user area), which instantiates it (caravel/verilog/rtl/user_project_wrapper
**Status: integration is paused, not complete.** The chip-level regression passes through gpu_caravel_top

| DCR addr | Name | Function |
|---|---|---|
| 0x00 | THREAD_COUNT | total threads in the launch (8-bit, reset 0) |
| 0x01 | PMU_RESET | one-cycle pulse: clear all PMU counters |
| 0x02 | PMU_SNAPSHOT | one-cycle pulse: latch counters into snapshot registers |
| 0x10–0x13 | PMU_CFG0–3 | 5-bit event selector for counter 0–3 (see Appendix A) |

| Wrapper signal | Connection |
|---|---|
| wb_clk_i, wb_rst_i | GPU clock and reset: one clock domain with the management So Wishbone. Timing is closed at 40 ns, so the Caravel core clock m stay at or below 25 MHz. |
| wbs_cyc_i … wbs_ack_o | the GPU's Wishbone slave; register and memory map as in §2.2.1 0x3000_0000, the user-project address window |
| user_irq[0] / [1] / [2] | done / fault (STATUS[4]) / 0 |
| io_oeb / io_out / la_data_out | tied to 1 / 0 / 0 inside gpu_caravel_top |
| la_data_in, la_oenb, io_in, analog_io, user_clock2 | unused |

Table 2.3: Caravel user-project connections.

with the tie-offs checked every cycle (`make MINI=1 top-caravel`). Hardening the wrapper did not finish: the template's macro-first method cannot reach the SRAM macros' metal-4 power pins inside a metal-1–4 sub-macro, and the flat wrapper run (`MINI=1 CARAVEL=1 ./run_asic.sh`) needed more than the 16 GB of the build machine. No Caravel-level simulation or shuttle precheck has been run. Caravel's own timing constraints also expose a hold requirement of about 3 ns on the Wishbone inputs (the management clock reaches the user area 4.65–5.57 ns late), which the flow can repair.

## 2.4 Address spaces

| Space | Size / width | Access | Notes |
|---|---|---|---|
| Program memory | 4 K words, 12-bit PC | fetch only | word-addressed; I-cache fetches 4-word blocks; only words 0..511 exist on chip (L15) |
| Global data | 16 K words, 14-bit address | LDR STR ATOM_* | only words 0..511 exist on chip (L15); the core faults accesses above 0xFFFF, but the bus carries only 14 bits, so 0x4000–0xFFFF **silently alias** (L14) |
| Shared memory | 64 words | LDSH STSH | per-core, shared by all 8 threads; bank = addr[1:0]; fault if address $\geq$ 64 |
| Host MMIO | 2 registers + 4 KB window | Wishbone | §2.2 |

Table 2.4: Address spaces of the ASIC profile.

## 2.5 Reset and launch sequence

1. Assert `reset`, then release.
2. Host writes CTRL with START=1 and THREAD_COUNT in bits [15:8]. START and the DCR write occur in the same bus cycle.
3. The dispatcher computes $\lceil$THREAD_COUNT/8$\rceil$ blocks. With one core, **blocks run sequentially**: each block resets the core and launches 8 threads (the last block may be partial); `R29` holds the block id.
4. When all blocks have finished, the dispatcher raises `flush_caches`; the D-cache invalidates and the L2 writes all dirty sectors back through the victim buffer. DONE is set only after every flush completes, so external memory is coherent when DONE reads 1.
5. DONE clears when the host writes START=0 and the dispatcher is idle. By code inspection (not simulated), holding START=1 after DONE re-triggers the flush loop, so software should drop START after observing

DONE.

Chapter 3

# Core microarchitecture

## 3.1 Pipeline

| Stage | Module(s) | Function |
|---|---|---|
| IF | `fetcher`, `icache` | Single-cycle fetch: the I-cache answers a hit combinationally for the PC of the warp the scheduler selects, and the instruction loads into the ID latch at the same clock edge (up to one instruction per cycle). A miss starts a line fill and issue waits. The fetcher holds nothing between cycles (until this revision it was a 3-state FSM taking 4 cycles per instruction). |
| ID | `decoder`, ID latch | Combinational decode; the `id_*` latch registers instruction, PC, warp id and active mask. Register-file read with write-through bypass. A flush of the previous occupant's warp squashes the latch only if no fresh instruction is loading in the same cycle. |
| EX | `alu`, `fp32_fma`, `pc` (per lane) | Per-lane integer ALU and FMA on the operands latched in ID (no forwarding network), NZP update, next-PC computation, branch/divergence/exception detection. |
| MEM | `lsu`, `shared_mem` | Memory instructions engage the LSU; the issuing warp parks in `WAITING_MEM` until its lanes complete. Other instructions pass through. |
| WB | `registers` | Three writers with fixed priority: FP writeback > LSU return > integer ALU. |

Table 3.1: Pipeline stages.

## 3.2 Datapath

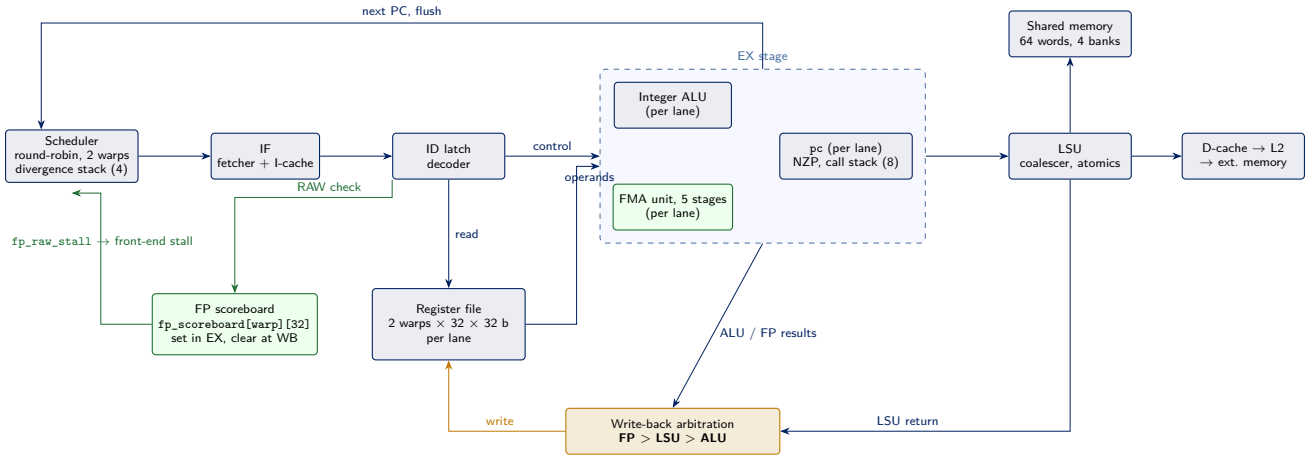



Figure 3.1: Core datapath. Blue: instruction/operand flow; green: FP hazard control; amber: write-back arbitration into the register file.

## 3.3 Warp scheduler

The scheduler (`scheduler.sv`) selects, every cycle and combinationally, the next `READY` warp with a non-empty active mask, round-robin from `rr_ptr`, excluding warps that are being flushed in the same cycle. Warp

states: `IDLE`, `READY`, `WAITING_MEM`, `WAITING_BARRIER`, `WAITING_FP`, `DONE_STATE`, `FAULTED`. Branch, call, divergence and exception outcomes are resolved in EX and redirect the front end through a one-cycle per-warp pulse `flush_warp_mask`; the squashed instructions are those between fetch and EX for that warp. Two issue rules keep a warp's instructions correct without forwarding (`core.sv`: `warp_in_flight`, `issue_hazard`):

- **Serializing instructions.** Branch, call, return, exit, `SYNC`, memory, shared-memory and atomic, FP and `DIV`/`MOD` instructions redirect or replay their warp. While one is in ID or EX its warp does not issue, so no wrong-path instruction is ever in flight. Plain ALU and constant instructions do not block.
- **Issue-time scoreboard.** A second decoder reads the source registers of the instruction fetched for the selected warp. If a same-warp instruction in ID or EX will still write one of them, the warp waits this cycle and the round-robin pointer moves to the next warp. A producer in MEM is written back while the consumer is in ID, which the register file's write-through covers.

NZP and ALU flags are written at the end of EX and read in EX, so `CMP` directly followed by `BRnzp` needs no wait. Measured on the mini profile (random programs, memory latency 6 cycles), this front end needs 1.23–1.32 times fewer cycles than the previous one; the remaining time is dominated by warps waiting on memory (40–50 % of cycles) and I-cache misses (19–26 %).

### 3.3.1 Divergence handling (SIMT stack)

Each warp has a 4-entry stack of `(PC, active_mask)` (`scheduler.sv:66-74`). In EX, the next PC of every active lane is computed. If all lanes agree, the warp continues. If they disagree, the lanes are split into two groups: group A (the group containing the lowest active lane) continues, and group B's `(target, mask)` is pushed.

- **Reconvergence** happens when a non-divergent branch of the running group targets *exactly* the PC on top of the stack: the entry is popped and the masks are merged.
- At `EXIT`, a non-empty stack is popped and the saved group resumes at its saved PC.
- There is **no** reconvergence at a join point or post-dominator. In practice lane groups run the remainder of the program serially until `EXIT`. Results are correct; a `SYNC` reached while the stack is non-empty faults (§4.5, erratum L1).

Maximum nesting is lane-count minus one (3 for four lanes), which fits the 4-entry stack. A profile with more than five lanes could overflow it (L7).

### 3.3.2 Call/return stack

Return addresses are held by each lane's `pc` instance: 8 entries per warp with a 3-bit pointer (`pc.sv:29-33`). `CALL` pushes `PC+1` and jumps to the absolute target; `RET_FN` pops. There is **no overflow or underflow detection**: the ninth nested call wraps the pointer and overwrites the oldest return address (erratum L6, demonstrated by `make limits`).

## 3.4 Floating-point unit

Each lane has a private `fp32_fma` with a fixed **5-cycle** latency from the EX stage to the register write. Pipeline

| Instruction | Internal operation $(a \cdot b) + c$ | Result |
|---|---|---|
| `FMA` | $a \cdot b + c$ with $a$=`rs`, $b$=`rt`, $c$=`rd` | `rs` × `rt` + `rd` |
| `FADD` | $a \cdot 1.0 + b$ | `rs` + `rt` |
| `FMUL` | $a \cdot b + 0$ | `rs` × `rt` |
| `FSUB` | $a \cdot 1.0 + (-b)$ | `rs` − `rt` |

stages: (1) mantissa multiply and exponent add, (2a) alignment, (2b) add/subtract with sign selection, (3) normalisation (leading-one detect), (4) pack. Numerics (this is *not* full IEEE-754; erratum L8):

- results are **truncated** (no rounding); denormal inputs read as zero and results below the normal range flush to ±0;

- exponent 255 is treated as an ordinary exponent (no NaN/Inf propagation); overflow produces an infinity *encoding*;
- the product exponent is computed even when an operand is zero, so $0 \times 2^{127} + c$ drops low mantissa bits of $c$;
- exact cancellation yields a zero carrying the sign of the product term $(a \cdot b)$.

### 3.4.1 Scoreboard and hazards

`fp_scoreboard[warp][reg]` is a 32-bit-per-warp vector. An FP instruction sets its destination bit when it fires in EX; the bit clears when the result reaches write-back (fifth cycle). The ID stage compares the instruction's `rs`, `rt` and (when read) `rd` with the scoreboard; a hit raises `fp_raw_stall`, which stalls the front end and squashes the EX latch for that cycle. In addition, when an FP instruction reaches EX the *warp* is parked in `WAITING_FP` and its younger instructions are flushed (`scheduler.sv:208-212`); it resumes when its FP result is written back. Other warps keep issuing. Integer results are covered by the issue-time scoreboard of the warp scheduler, loads by `WAITING_MEM`. In every simulation (`MAHI_SIM_CHECKS`) an assertion checks each cycle that no instruction in EX reads a register a MEM, WB or LSU result is still producing; with the scoreboard disabled every random program trips it.

### 3.4.2 Register-file write arbitration

Priority 1 FP write-back, priority 2 LSU return, priority 3 integer ALU (`registers.sv`); lower priorities are suppressed when the same warp *and* register collide. Writes to `R29`–`R31` are ignored by all writers.

**Why the ALU-versus-FP collision guard cannot be exercised**

Removing the ALU-side FP-collision guard in a mutation experiment was *not* detected by 100 random programs. The design reason is that an FP operation parks its own warp in `WAITING_FP` and flushes its younger instructions, while older instructions have left the pipeline before the five-cycle FP result returns, so an ALU write to the same warp/register cannot coincide with the FP write-back. This is a design argument supported by the 0/100 mutation survival; it has **not** been formally proven.

## 3.5 Memory subsystem

### 3.5.1 Load/store unit and coalescing

The LSU (`lsu.sv`) accepts one memory instruction per warp and works through the active lanes with a state machine (`IDLE, COALESCE_READ, WAIT_READ, COALESCE_WRITE, WAIT_WRITE, WAIT_SHARED, ATOMIC_*`).

- **Reads:** the lowest pending lane selects a 128-bit block; every pending lane whose word lies in that block is served by the same transaction. Lanes in other blocks need further transactions.
- **Writes:** all pending lanes that map to the selected block are packed into one 128-bit beat with a 4-bit word strobe, so contiguous lane addresses produce a single block write.
- **Atomics** (`ATOM_ADD`, `ATOM_CAS`) are serialised lane by lane: read the block, apply the operation, write back one word, return the old value in `rd`.
- The effective address is `rs + sign_extend(imm16)` for `LDR`/`STR`; atomics and shared-memory accesses use `rs` alone.

### 3.5.2 Banked shared memory

64 words in 4 banks of 16 words: `bank = addr[1:0]`, `offset = addr[5:2]`. Every lane has its own request port. Per cycle, lanes are processed in index order and a lane is deferred (not dropped) if

- *write bank conflict*: a lower lane writes the same bank in the same cycle;
- *read bank conflict*: a lower lane reads the same bank at a *different* address;
- *read-after-write*: a lower lane writes the address being read.

Lanes reading the *same* address as a lower lane are served by **multicast broadcast** (they copy the data fetched for that lower lane). Shared memory resets to zero. The memory is shared by all warps of the core; ordering between warps requires `SYNC`.

### 3.5.3 Caches and victim write buffer

| Level | Organisation | Policy |
|---|---|---|
| I-cache | 16 lines, direct-mapped, 4-word lines | read-only; miss fetches the block from the program controller |
| D-cache (L1) | 16 lines = 8 sets × 2 ways, LRU bit | **write-through**; write hit merges into the line, write miss does not allocate; read miss allocates |
| L2 | 64 lines = 16 sets × 4 ways, 3-bit PLRU, 4 sectors of 32 bit per line with per-sector dirty bits | **write-back, write-allocate**; evicting a dirty line pushes it into the victim write buffer |
| Victim write buffer | 4 entries (128-bit data + 4 dirty bits) | merges pushes to the same address (dirty-sector merge); drains when half full, on a flush, or after an idle timeout (31 cycles); an L2 miss that hits the buffer is served from it and the entry is popped (probe-hit bypass) |

Table 3.2: Cache hierarchy (ASIC profile).

The L2 must not push to a full buffer: a push is attempted only when the buffer reports space *and* no push is already in flight (this handshake was a bug, B4, Chapter 5); the buffer additionally asserts an error if a push ever arrives while full. At the end of a launch the D-cache is invalidated and the L2 flushed, after which external memory holds all results.

## 3.6 Exceptions

Division by zero (cause 1), out-of-range addresses (global >`0xFFFF`, shared ≥64; cause 2) and a `SYNC` executed while the warp's divergence stack is non-empty (cause 3) mark the offending warp `FAULTED` and flush it. The other warps continue and a faulted warp counts as finished for DONE. The first fault is reported to the host: STATUS[4] and `gpu_fault_pin` go high and FAULT_INFO holds its cause, warp and PC (§2.2.1); write 1 to STATUS[4] to clear. With several cores only the lowest-numbered faulting core is reported.

## 3.7 Performance monitoring unit

Four 32-bit counters, each selecting one of 32 event lines (Appendix A) through DCR 0x10–0x13, with reset and snapshot pulses. L2 hit/miss events read zero (L4), and the PMU registers are not reachable from the Wishbone interface (L13).

Chapter 4

# Instruction set architecture

## 4.1 Instruction formats

All instructions are 32 bits. The opcode occupies bits [31:26].

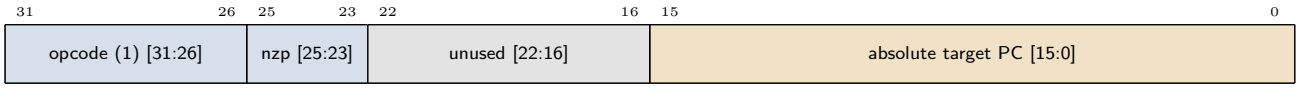
### R-type (register–register)




### I-type (immediate, memory)




### Branch type (`BRnzp`, `CALL`)




`CALL` uses opcode 13 with bits [25:16] unused. `RET_FN`, `EXIT`, `SYNC` and `NOP` carry only the opcode.

### Load-upper-immediate (`LUI`)




`rd = imm20 « 12`.

**Operand-slot conventions**

For `STR` and `STSH` the register holding the *value to store* is encoded in the `rd` field and the address register in `rs` (the decoder re-routes the `rd` slot into the second source). For `MAC`, `FMA` and `ATOM_CAS`, `rd` is also an input. `BRnzp` encodes the condition in `[25:23]` as `N=bit 25, Z=bit 24, P=bit 23`.

## 4.2 Register file conventions

Each lane holds 2 warps × 32 = 64 registers. The register file is written by up to three sources per cycle (Chapter 3); a write-through bypass forwards a value to a same-cycle read of the same warp.

## 4.3 Instruction reference

*Notation:* `rd, rs, rt` are registers, `imm` a 16-bit signed immediate, `T` an absolute program address. Integer values are 32-bit two's complement unless stated.

| Register | Name | Access | Value / convention |
|---|---|---|---|
| R0–R28 | general purpose | R/W | reset to 0; no ABI is imposed by the hardware |
| R29 | `blockIdx` | R, writes ignored | block id assigned by the dispatcher (8 bits, zero-extended) |
| R30 | `blockDim` | R, writes ignored | `THREADS_PER_BLOCK` × `NUM_WARPS` = **8** (threads per block) |
| R31 | `threadIdx` | R, writes ignored | `warp_id` × 4 + `lane` (0–7) |

Table 4.1: Per-thread register file. Global thread id: `MUL R0,R29,R30 ; ADD R0,R0,R31`.

### 4.3.1 Integer arithmetic and logic

| Mnemonic | Op | Operands | Behaviour |
|---|---|---|---|
| ADD | 3 | rd, rs, rt | $rd = rs + rt$ (wraps) |
| SUB | 4 | rd, rs, rt | $rd = rs - rt$ (wraps) |
| MUL | 5 | rd, rs, rt | low 32 bits of the signed product |
| DIV | 6 | rd, rs, rt | unsigned quotient; `rt=0` faults. Multi-cycle: shared iterative divider, up to 33 cycles per active lane (R6) |
| MOD | 22 | rd, rs, rt | unsigned remainder; same caveats as `DIV` |
| AND | 17 | rd, rs, rt | bitwise AND |
| OR | 18 | rd, rs, rt | bitwise OR |
| XOR | 19 | rd, rs, rt | bitwise XOR |
| SHL | 20 | rd, rs, rt | logical left shift by `rt`; counts $\geq 32$ give 0 |
| SHR | 21 | rd, rs, rt | logical right shift; counts $\geq 32$ give 0 |
| MIN | 23 | rd, rs, rt | signed minimum |
| MAX | 24 | rd, rs, rt | signed maximum |
| ABS | 25 | rd, rs | signed absolute value (`0x80000000` $\mapsto$ `0x80000000`) |
| NEG | 26 | rd, rs | two's-complement negate |
| MAC | 27 | rd, rs, rt | $rd = rd + rs \cdot rt$ (signed, low 32 bits); reads `rd` |
| POPCNT | 30 | rd, rs | number of set bits |
| CLZ | 31 | rd, rs | leading zeros (CLZ(0) = 32) |
| BREV | 32 | rd, rs | bit reversal |
| CONST | 9 | rd, imm | $rd = \text{sext}(imm16)$ |
| LUI | 29 | rd, imm20 | $rd = imm20 \ll 12$ |

The ALU also computes carry, overflow, zero, negative and saturation flags, but **no instruction reads them**.

### 4.3.2 Floating point (single precision)

| Mnemonic | Op | Operands | Behaviour |
|---|---|---|---|
| FADD | 33 | rd, rs, rt | $rd = rs + rt$ |
| FSUB | 34 | rd, rs, rt | $rd = rs - rt$ |
| FMUL | 35 | rd, rs, rt | $rd = rs \times rt$ |
| FMA | 36 | rd, rs, rt | $rd = rs \times rt + rd$ (reads `rd`); single rounding point: truncation |

### 4.3.3 Compare, branch and call

| Mnemonic | Op | Operands | Behaviour |
|---|---|---|---|
| CMP | 2 | rs, rt | sets this lane's NZP flags: `N = rs<rt, Z = rs==rt, P = rs>rt` using an **unsigned** comparison (L5). Writes no register |

| Mnemonic | Op | Operands | Behaviour |
|---|---|---|---|
| BRnzp | 1 | nzp, T | taken iff (NZP & nzp) $\neq 0$. NZP is 0 after reset, so a branch before any CMP falls through even with nzp=111 |
| CALL | 13 | T | push PC+1 on the lane's return stack, jump to T |
| RET_FN | 14 | — | pop the return stack and jump (undefined if empty) |
| EXIT | 15 | — | terminate the lane; if the warp's divergence stack is non-empty the next group resumes |
| NOP | 0 | — | no operation |

### 4.3.4 Memory, shared memory and atomics

| Mnemonic | Op | Operands | Behaviour |
|---|---|---|---|
| LDR | 7 | rd, [rs+imm] | load word from global memory; fault if address >0xFFFF; addresses ≥0x4000 alias (L14) |
| STR | 8 | rv, [rs+imm] | store rv (encoded in the rd slot) to global memory |
| LDSH | 11 | rd, [rs] | load from shared memory (no offset); fault if address ≥ 64 |
| STSH | 12 | rv, [rs] | store rv (rd slot) to shared memory |
| ATOM_ADD | 16 | rd, rs, rt | old = mem[$rs$]; mem[$rs$] = old+$rt$; $rd$ = old; lanes serialised |
| ATOM_CAS | 28 | rd, rs, rt | old = mem[$rs$]; if old = $rd$ then mem[$rs$] = $rt$; $rd$ = old (rd is both compare input and result) |

### 4.3.5 Synchronisation

| Mnemonic | Op | Operands | Behaviour |
|---|---|---|---|
| SYNC | 10 | — | block barrier (__syncthreads()): the warp waits until every non-finished warp of the core has executed SYNC. **Requires warp-uniform control flow** (§4.5) |

Opcodes 37–63 are reserved and decode as inert instructions.

## 4.4 Programming example

The following program is the instruction sequence of a branch/call test; the encodings were generated by the project's assembler functions (scripts/isa.py) and executed on the RTL.

```
0:  24200007  CONST R1, 7
1:  145f0800  MUL   R2, R31, R1      ; R2 = threadIdx * 7
2:  08020800  CMP   R2, R1           ; N/Z/P = R2 <,==,> R1 (unsigned)
3:  04800005  BRp   5                ; skip the call when R2 > R1
4:  34000007  CALL  7
5:  205f012c  STR   R2, [R31+300]    ; mem[300+tid] = R2
6:  3c000000  EXIT
7:  0c420800  ADD   R2, R2, R1       ; callee
8:  38000000  RET_FN
```

## 4.5 Architectural restrictions and hazards

**R1 — `SYNC` requires warp-uniform control flow**

The barrier is tracked at **warp** granularity: when a lane group executes `SYNC`, the warp enters `WAITING_BARRIER` and is counted once (`barrier_count >= num_active_warps_comb`, `scheduler.sv:287`). Because divergent lane groups are serialised until `EXIT` (no join-point reconvergence), a `SYNC` that follows divergent code is executed once *per group*, and the first group to arrive releases the barrier while other lanes have not yet performed their stores. This used to let threads read stale shared memory (observed: 4 of 8 threads in the directed test, and in 18 of 240 early random programs); the hardware now detects it and faults the warp with cause 3 instead (the kernel is rejected, not silently wrong). **Rule:** every branch and loop bound that precedes a `SYNC` must be uniform across the warp (derived from constants, `R29`, `R30`, or the warp id); it may differ *between* warps. Divergence without barriers is fully supported. A `SYNC` executed after some lanes have already executed `EXIT` is likewise unsupported.

**R2 — return-stack overflow**

Nesting deeper than 8 `CALL`s wraps the 3-bit stack pointer silently and corrupts the oldest return address; a ninth nested call hangs or mis-returns on the RTL. `RET_FN` with an empty stack jumps to a stale address. Keep call depth $\leq 8$.

**Other rules**

**R3** `CMP` is unsigned. **R4** A conditional/unconditional `BRnzp` needs a preceding `CMP` (NZP resets to 0); an "always" jump is `CMP` of a register with itself followed by `BR 111`. **R5** Do not write `R29`–`R31`. **R6** `DIV`/`MOD` are slow (shared iterative divider: up to 33 cycles per active lane, the warp waits) and need a non-zero divisor. **R7** Division by zero, bad addresses and `SYNC` while diverged stop the warp; check STATUS[4] after DONE. **R8** Results are guaranteed in external memory only after DONE. **R9** Do not rely on the old value returned by `ATOM_ADD` being a particular order across lanes.

Chapter 5

# Verification and physical implementation

## 5.1 Verification methodology

1. **Lint:** Verilator `-Wall` on the sim, ASIC and FPGA profiles; ten baseline warning classes are waived, any new class fails.
2. **Unit tests** (Verilator, self-checking): ALU (56 736 vectors, run with and without `USE_COMB_DIV`), decoder (all 64 opcodes), `pc`, register file, FMA (4 013 pipelined results against a double-precision reference), D-cache and L2+VWB (random partial-write traffic with random memory latency against a golden memory).
3. **Directed system tests:** NZP/branch/nested-call test on the ASIC profile; 2-core, 32-thread 5×5 matrix multiply (CALL/RET, divergence, DIV).
4. **SVA checkers** bound into the RTL (no source edits): issued PC equals fetched PC, no dropped instruction, return-stack pointer discipline, CMP updates NZP, divergence-stack bound, LSU strobe sanity, fetcher overlap, VWB no-push-while-full.
5. **Golden reference model and constrained-random fuzzing.** A Python interpreter (`Machine` in `isa.py`) executes the same machine words, including a bit-exact port of the FMA datapath. The generator (`gen.py`) emits terminating, race-free programs using structured control flow (forward if/else, loops with per-thread or per-warp trip counts, an acyclic call graph), integer and FP arithmetic, global and shared memory, `SYNC` barriers and `ATOM_ADD`; each program dumps all registers to memory. The runner (`rand_regress.py`) executes each program on the RTL with randomised memory latency and compares *all* 1024 data words with the model. Programs that use barriers are generated in a "uniform" mode (restriction R1) with deliberate warp imbalance.
6. **Limitation characterisation** (`make limits`) and a **documentation consistency check** (`make docs-check`) keep this manual and the repository synchronised.

`make verify` runs lint, unit, system, a short random run, limitation and documentation checks.

## 5.2 Regression results

| Campaign (generator content) | Memory latency (max extra cycles) | Programs / latency | Mismatches |
|---|---|---|---|
| Integer, control flow, CALL/RET | 0, 2, 5, 10, 25 | 600 each (3000) | 0 |
| + FP, shared memory, SYNC, atomics | 0, 5, 20 | 1200 each (3600) | 0 |
| + warp-imbalanced barriers | 0, 3, 10, 30 | 800 each (3200) | 0 |
| Same generator, **current RTL** (L2 data in SRAM) | 0, 3, 10, 30 | 750 each (3000) | 0 |
| Same generator, current RTL, other seeds | 0, 2, 5, 10, 25, 40 | 1700 each (10200) | 0 |

Table 5.1: Constrained-random campaigns (ASIC profile, 8 threads). The first three ran before the L2 data array moved into an SRAM macro; the last two ran on the current RTL. Each program is 150–300 instructions.

All 22 800 runs matched the model bit-for-bit with all SVA checkers enabled. The first campaign pre-dates the FP/shared-memory/atomic extension of the generator; the latency sweep requested for sign-off (0, 2, 5, 10, 25) was therefore also covered by the later campaigns, and the last campaign repeats it on the current RTL. At chip level (`make top-quick`) 40 seeds, two kernels each with a soft reset between them, ran purely through the Wishbone pins and matched the model; the same test passes against the real OpenRAM model (`make top-macro`), and a zero-delay gate-level simulation of the synthesised netlist matched for 200 seeds (`make gls`; pre-placement, no SDF, two-state). Mutation scores were not re-measured after the L2 change.

## 5.3 Mutation testing

Faults were injected into copies of the RTL and the random regression was run on 100 programs at 4 cycles of latency (200 for the barrier fault). The figure is the number of programs whose result differed from the model. All faults were detected except those marked. Additional ALU, decoder, register-file and FMA mutants were killed

| Injected fault | Detected by | Comment |
|---|---|---|
| `FSUB` sign not inverted | 18 / 100 | also unit test |
| FP normalisation exponent off by one | 54 / 100 | also unit test |
| `ATOM_ADD` adds a constant instead of the operand | 55 / 100 | |
| Shared-memory store uses lane 0's data for all lanes | 21 / 100 | |
| Barrier released when the first warp arrives | 18 / 200 | **0 / 100** before warp-imbalanced workloads were added |
| Register-file ALU-vs-FP collision guard removed | 0 / 100 | survived; design argument in Chapter 3 |
| FMA alignment threshold 72→40 | not detected | effect lies below the 24-bit truncation (unobservable) |
| Original bugs re-introduced (B1–B4): dead NZP/CALL/RET; ID-latch drop; fetcher/warp skew; L2 push-while-full | 98 / 92 / 98 / 100 of 100 | each also caught by a unit or system test and by a specific SVA |

Table 5.2: Mutation-testing scorecard.

by their unit tests (e.g. overflow flag, CLZ direction, signed MIN, STR operand slot, immediate sign-extension, FP priority, r30/r31 protection).

## 5.4 Bugs found and fixed during verification

| ID | Fix commit | Summary |
|---|---|---|
| B1 | `499a7d0` | ASIC port deleted the NZP, return-address and stack-pointer updates in `pc.sv`; branches and calls were dead. |
| B2 | `0ab48c4` | ID-latch squash for the previous occupant's warp discarded a fresh instruction of another warp; the fetcher had already consumed it. |
| B3 | `ab57e33` | Fetcher held an instruction for one warp's PC while the scheduler issued another warp: wrong instruction issued; fixed with a PC tag and refetch. |
| B4 | `e086fc4` | L2 pushed to a full victim write buffer (stale ready); the dirty line was dropped. |

Table 5.3: Fixed bugs. B2–B4 are latent in the shared design and are fixed in the ASIC copy only (erratum L10).

## 5.5 Physical implementation

**Status — the mini profile is routed with per-corner SRAM timing; the full profile is not**

**RTL revision.** The layout below was built from the RTL before the single-cycle fetch and the removal of operand forwarding (§3); a layout of the current RTL is in progress. Pre-placement timing of the current RTL no longer has core-datapath paths among the worst families (the SRAM half-cycle reads lead).
The **mini profile** (`MINI=1`: 2 lanes × 2 warps, 32 words of shared memory) has a routed layout at the 40 ns clock (`flow/runs/place_1029_1604`) with: routing DRC 0, antenna 0, LVS 0 (macros compared at their pins), KLayout DRC 0, and a full-chip Magic DRC with 0 integration errors (the OpenRAM macros' own 12 984 239 violation boxes are waived, see below). Setup is met with $\geq +3.27$ ns and hold with $\geq +0.27$ ns in all nine corners, using SRAM liberty files characterised with OpenRAM at the TT, SS and FF corners. The Magic-versus-KLayout XOR shows 20 differences, all on the cell-boundary marker layer 235/4 (not a mask layer). **Not done:** the full-size profile has never been routed (it needs a rectangular die inside the 2.92 mm × 3.52 mm Caravel user area and probably more than 16 GB of memory); the macros are black boxes in LVS; the SRAM read-port (`dout1`) nets exceed the macro's own 27.6 fF capacitance limit (36–39 pins; slow-corner slew up to 6 ns, extrapolated; waiver recorded; erratum L17), and 642 capacitance and 524 fanout violations remain; there is no simulation with SDF delays, no formal RTL-to-netlist proof, and no Caravel wrapper. The routed netlist passes zero-delay gate-level simulation and is formally equivalent to the synthesized netlist, and the SRAM macro passes transistor-level LVS.

### 5.5.1 Flow and configuration

`run_asic.sh` converts the SystemVerilog to Verilog with `sv2v` (`-DSYNTHESIS -DTARGET_ASIC=1`, plus `-DTARGET_ASIC_MINI=1` for the mini profile), runs a Yosys elaboration/latch audit (zero latches required), then runs LibreLane 3.0.16 (OpenROAD-based) in a container with `flow/config.yaml` (`MINI=1 ./run_asic.sh` reproduces the reference result: 85 of 85 compared metrics identical to the hand-assembled first run):

| Parameter | Value |
|---|---|
| Design / top | `gpu_asic_top` |
| Clock | port `clk`, 40.0 ns, 0.25 ns uncertainty, 5 % OCV derate; clock source modelled as `clkbuf_16` |
| Die / core area (mini) | [0,0,2400,2400] µm / [50,50,2350,2350] µm, absolute sizing; placement density target 30 % |
| PDK / library | `sky130A` (version pinned in the Makefile) / `sky130_fd_sc_hd` |
| Synthesis | `SYNTH_STRATEGY "DELAY 4"` (the only strategy whose netlist met 40 ns before placement; +7.8 % area) |
| Macros | three `sky130_sram_2kbyte_1rw1r_32x512_8` (PMEM window, DMEM window, L2 data array), per-corner liberty (TT, SS, FF, characterised with OpenRAM); power connections `VPWR VGND vccd1 vssd1`; 12 µm metal-4 keep-out bands at the macro pin edges while the power grid is generated |
| Repair and timing | post-routing slew/cap repair (margins 70 %/70 %, 1 200 µm wire-length cap); post-CTS hold margin 0.4 ns; timing repair again after the slew repair (hold margin 0.4 ns); antenna repair up to 8 rounds at 25 % |
| Stream-out and DRC | KLayout GDS is the deliverable (one top cell, `KLAYOUT_CONFLICT_RESOLUTION OverwriteCell`); flow Magic DRC non-fatal, verdict by `make signoff-drc` |

### 5.5.2 Results of the final mini layout (`flow/runs/place_1029_1604`)

The OpenRAM macros are not clean under Magic's sky130 rules (5 579 171 boxes in 31 rule types on one macro alone). A box is waived only if its rule also fires on the standalone macro and it lies inside a macro outline; whether a shuttle accepts these macros is the operator's decision (erratum L17 and the project's open-problems list). KLayout's deck reports no violation on the macro.

| Metric | Value |
|---|---|
| Instances | 987 884 in total: 205 333 standard cells (1.61 mm$^2$), 62 490 tap cells, 782 548 fill cells, 3 macros (0.85 mm$^2$); of the standard cells 14 732 flip-flops, 7 187 antenna diodes, 3 069 clock buffers |
| Die / utilisation | 2.4 mm × 2.4 mm; standard-cell utilisation 36 % |
| Routing | 9.36 m of wire, 1 050 777 vias, 0 DRC errors, 0 antenna violations |
| LVS / XOR / KLayout DRC | 0 / 20 (layer 235/4 boundary marker only) / 0 |
| Magic DRC, full chip | 12 984 239 boxes, all inside the three macros and in rules that also fire on the standalone OpenRAM macro; **0 integration errors** (`make signoff-drc`) |
| Slew, slow corner | 6 300 pins above the flow's 0.75 ns target, 278 above the library's 1.5 ns range, 128 above 2 ns (worst 6.0 ns, on SRAM `dout1` outputs at loads beyond the library table); clock-root input about 1 ns |
| Capacitance / fanout violations | 642 / 524 (open); 36–39 are SRAM `dout1` pins above the macro's 27.6 fF limit (waived) |
| IR drop (worst / average) | 0.9 mV / 19 µV |
| Power (vectorless estimate) | 39.9 mW |

### 5.5.3 Static timing across PVT corners

Worst slack from OpenSTA on the routed design (clock 40 ns, 0.25 ns uncertainty, 5 % derate). Corner names: `max/nom/min` = parasitic extraction corner, `ss/tt/ff` = process/voltage/temperature.

| Corner | Process | V / T | Setup WS (ns) | Hold WS (ns) |
|---|---|---|---|---|
| `max_ss_100C_1v60` | slow | 1.60 V, 100 °C | **3.274** (worst setup) | 1.462 |
| `nom_ss_100C_1v60` | slow | 1.60 V, 100 °C | 3.932 | 1.486 |
| `min_ss_100C_1v60` | slow | 1.60 V, 100 °C | 4.554 | 1.509 |
| `max_tt_025C_1v80` | typical | 1.80 V, 25 °C | 13.624 | 0.565 |
| `nom_tt_025C_1v80` | typical | 1.80 V, 25 °C | 13.693 | 0.593 |
| `min_tt_025C_1v80` | typical | 1.80 V, 25 °C | 13.829 | 0.604 |
| `max_ff_n40C_1v95` | fast | 1.95 V, −40 °C | 15.753 | **0.271** (worst hold) |
| `nom_ff_n40C_1v95` | fast | 1.95 V, −40 °C | 15.830 | 0.297 |
| `min_ff_n40C_1v95` | fast | 1.95 V, −40 °C | 15.940 | 0.309 |

Table 5.4: Slack by corner of the final mini layout. The SRAM macros are timed with liberty files characterised per corner (TT, SS, FF) with OpenRAM; the earlier typical-only library was 4.7 times too fast (erratum L17).

### 5.5.4 Outstanding sign-off items

The shuttle's decision on the macro DRC waiver; the SRAM `dout1` output nets above the macro's capacitance limit, and the other capacitance and fanout violations; gate-level simulation with the SDF files the flow writes, a formal RTL-to-netlist proof; a routed full-size profile; the Caravel wrapper, harness simulation and precheck.

Chapter 6

# Errata and known limitations

Each entry has a characterisation test or static check in the repository (`make limits`, `make docs-check`); a change in behaviour fails those checks so this chapter is updated. Severity: **H** may give silently wrong results or blocks use, **M** surprising but contained, **L** inconvenience.

| ID | Sev | Title | Description / workaround |
|---|---|---|---|
| L1 | M | No join-point reconvergence | Lane groups run the rest of the program serially; a `SYNC` after divergence faults (cause 3) instead of releasing early. Use warp-uniform control flow before every barrier (R1). |
| L2 | – | `DIV`/`MOD` stubbed | **Resolved:** computed by a shared iterative divider (R6). |
| L3 | – | Memory at the chip top (resolved) | Previously tied off. The window at `0x3000_1000` is implemented on two OpenRAM macros; see L15–L17 for what remains. |
| L4 | L | L2 PMU events zero | L2 hit/miss/evict event outputs are constants. |
| L5 | M | `CMP` unsigned; NZP resets to 0 | See rules R3, R4. |
| L6 | M | 8-deep call stack wraps | Silent corruption on the ninth nested call. |
| L7 | L | 4-deep divergence stack | Cannot overflow with four lanes; could with $\geq 6$ lanes. |
| L8 | M | FP is not IEEE-754 | Truncation, flush-to-zero, no NaN/Inf, zero-operand alignment quirk. |
| L9 | M | Atomics coverage | `ATOM_CAS`, ordered old values and multi-core coherence untested by random tests. |
| L10 | M | Fixes only in ASIC copy | B2–B4 are not applied to the generic `src/rtl`. |
| L11 | L | Test debt | Legacy testbenches do not build against the current block interface; `tb_matmul` times out under iverilog (passes under Verilator). |
| L12 | – | Faults invisible | **Resolved:** STATUS[4], FAULT_INFO and `gpu_fault_pin` report the first fault. The faulted warp's later results are still missing; check STATUS[4] after DONE. |
| L13 | L | PMU/DCR unreachable | Only DCR address 0 is driven by the Wishbone wrapper. |
| L14 | M | Address aliasing | The fault limit (`0xFFFF`) exceeds the 14-bit bus: accesses in `0x4000`–`0xFFFF` wrap onto `addr & 0x3FFF` without a fault. Keep global addresses below `0x4000`. |
| L15 | M | Only 512+512 words on chip | GPU program and data words $\geq$512 read as 0 (instruction fetch) or read 0 / drop stores and set STATUS[2] (data). Programs must fit 512 words; the random generator uses a 32-word private region per thread. |
| L16 | M | Host access exclusive; soft reset to re-run | Window accesses are refused while BUSY. Caches are flushed but not invalidated after a run, so write CTRL[1] (soft reset) before loading a new kernel. |
| L17 | M | SRAM read-port outputs exceed the macro's capacitance limit | Per-corner liberty files now exist (the shipped typical-only file was 4.7 times too fast, clock to output 0.38 ns against 1.81 ns), and timing is met in all nine corners. After routing, 36–39 `dout1` pins carry 40–70 fF against the macro's 27.6 fF limit; the slow-corner slew there reaches about 6 ns (extrapolated). Waiver recorded, not fixed. |

*Pin count:* the wrapper has 108 signal pins; meeting a smaller GPIO budget (for example a 38-pin user area) would require a narrower address/data interface.

## Appendix A

# PMU event map

The 32-bit event bus (`core.sv`, `event_bus`) feeds four counters; each counter selects one bit with a 5-bit selector.

| Bit | Event | Bit | Event |
|---|---|---|---|
| 3 | cycle (core running) | 16 | scheduler stall: barrier |
| 4 | active (EX has active lanes) | 17 | scheduler stall: no ready warp |
| 5 | instruction issued | 18 | D-cache read access |
| 6 | fetch stall | 19 | D-cache read hit |
| 7 | flush | 20 | D-cache read stall |
| 8 | memory operation started | 21 | D-cache write access |
| 9 | I-cache access | 22 | D-cache write hit |
| 10 | I-cache hit | 23 | D-cache write stall |
| 11 | I-cache stall | 24 | FP instruction fired |
| 12 | scheduler idle | 25 | FP write-back valid |
| 13 | warp switch | 26 | FP RAW stall |
| 14 | divergence | 27 | FP write-back collision |
| 15 | scheduler stall: memory | 0–2, 28–31 | unused (0) |

## Appendix B

# Provenance and how to re-check this document

Source: repository commit `dadec7f` (RTL under `ASIC/src/rtl`, flow under `ASIC/flow`); physical numbers from `ASIC/flow/runs/RUN_2026-10-19_14-54-17`. The repository documentation set in `ASIC/docs` (pages 01–08) contains line-level citations for every statement here. To re-audit: run `make verify` in `ASIC/`; `make docs-check` validates generated tables and citations; `make limits` validates Chapter 6; `make datasheet` rebuilds this PDF.